MANIPAL INSTITUTE OF TECHNOLOGY, BENGALURU
Department of Computer Science and Engineering

OntoRisk BI

Ontology-Driven Semantic Data Warehouse for Proactive Business Risk Intelligence

PROJECT EXPLANATION & TECHNICAL GUIDE

What was built, how every technology is used and why, the results, how to run it, and answers to the questions examiners usually ask

Team member	Register No.
Aishwarya Patil	235890068
Thiriveedhi Venkata Subba Sai Teja	235890021
Dhruv Naveen	235890154
Shaswat Das	235890222

Section B · B.Tech Computer Science and Engineering
Guide: **Dr. Sathiyaraj Rajendran**, Associate Professor, CSE Dept., M.I.T Bengaluru

Contents

1. The project in one page **4**

What was delivered . . . 4

2. Requirement traceability — every item in the report, and where it is **5**

3. Architecture **6**

Folder structure . . . 6

4. How each stage works **7**

4.1 Data collection — the corpus . . . 7

4.2 Preprocessing and named entity recognition . . . 7

4.3 The co-occurrence relation rule . . . 8

4.4 The ontology . . . 9

4.5 SPARQL validation . . . 10

4.6 The warehouse (DuckDB star schema) . . . 11

4.7 Data mining . . . 12

4.8 Evaluation of the extraction layer . . . 15

4.9 The dashboard . . . 16

5. Technologies — what, how it is used here, why it matters **17**

Python 3 . . . 17

pandas . . . 17

spaCy (en_core_web_sm) . . . 17

RDF and RDFLib . . . 17

OWL ontology and Protégé . . . 18

SPARQL . . . 18

DuckDB . . . 18

Star schema, Kimball and Inmon . . . 19

OLAP operations . . . 19

mlxtend — FP-Growth and Apriori . . . 19

scikit-learn — k-Means, silhouette, PCA . . . 19

Streamlit and Plotly . . . 20

pytest, matplotlib, Git/GitHub . . . 20

6. Methodologies in more depth **21**

6.1 The semantic-web stack . . . 21

6.2 ETL and data quality . . . 21

6.3 Rule-based relation extraction vs. learned extraction . . . 21

6.4 Association-rule mining, step by step . . . 21

6.5 Clustering, step by step . . . 21

6.6 Evaluation metrics . . . 21

7. Results summary **22**

8. How to run the project **23**

8.1 One-time setup . . . 23
8.2 Build everything . . . 23
8.3 Test, then open the dashboard . . . 23
8.4 Optional steps . . . 23
8.5 Troubleshooting . . . 23
9. Team allocation and timeline (Table 4.4, mapped to code) 24
10. Limitations and future scope 24
Limitations (from the report, confirmed by the build) . . . 24
Future scope . . . 24
11. Likely questions and model answers 25
12. Glossary 26
13. References 27

1. The project in one page

Elevator pitch. Businesses learn about risks — a flood at a warehouse, congestion at a port, a new export ban, a supplier going bust, a product recall — from the news, but that news is unstructured text that BI tools cannot query. OntoRisk BI reads news articles, finds *which organisation* is affected by *what kind of risk where*, stores that knowledge twice — as RDF triples in a small ontology (for meaning and SPARQL) and as rows in a DuckDB star schema (for fast OLAP) — then mines it for co-occurring risk factors (FP-Growth) and groups of similar disruptions (k-Means), and shows everything on one Streamlit dashboard.

The problem. SMEs in supply chain, retail, finance and manufacturing have no affordable way to turn risk-relevant text into structured, queryable intelligence. Text-mining outputs usually live apart from the warehouse, and semantic-web tools are rarely combined with star-schema/OLAP practice.

The contribution. As the report states, the contribution is **integration, not novelty in any one layer**: semantic layer + dimensional warehouse + data-mining layer in a single, open, reproducible pipeline, each layer in the simplest form that still produces a genuine artefact for the next one, with an explicit Indian context (monsoon disruption, port congestion, domestic policy change).

What was delivered

Deliverable	Where	Result of the latest run
Curated news corpus (150–300 articles)	data/raw/news_corpus.csv	243 raw records → 240 clean articles
Compact risk ontology (6 classes, 2 object properties)	ontology/ontorisk.owl (Protégé)	6 classes, 2 object properties, 19 data properties
spaCy NER + risk vocabulary tagging	src/ontorisk/nlp.py	1590 entity / term mentions
Co-occurrence RDF triples linked to ontology	ontology/ontorisk_kg.ttl	6,068 triples (5,605 generated)
SPARQL validation queries	outputs/sparql_validation.json	6/6 passing
DuckDB star schema + semantic bridge	warehouse/ontorisk.duckdb	387 fact rows, 0 orphan keys
FP-Growth association rules (Apriori cross-check)	outputs/association_rules.csv	103 frequent itemsets, 27 rules
k-Means clustering (elbow + silhouette)	outputs/clusters.csv	k = 5, silhouette = 0.239
Triple precision evaluation	outputs/triple_review_sample.csv	precision 72.4%, recall 92.6%
Single-page Streamlit dashboard	dashboard/app.py	4 KPIs, OLAP filters, rules table, cluster scatter
Automated tests	tests/	17 pytest tests (16 + 1 that needs Streamlit)

2. Requirement traceability — every item in the report, and where it is

The table walks through the report (abstract, objectives, Chapter 4 and the scope decisions) and points to the code that satisfies each point. Use it when someone asks *“where did you implement X?”*.

Report requirement	Implementation	Status
Obj. 1 — small domain ontology in Protégé: RiskEvent, Entity, Location, RiskType + supporting classes; properties “affects”, “located-in”	ontology.py generates ontorisk.owl (RDF/XML, opens in Protégé) and .ttl. Supporting classes: SourceArticle, RiskTerm. Impact & provenance are data properties, as in §4.4	Done
Obj. 2 — spaCy NER on news, risk terms, RDF triples linked to ontology	nlp.py (en_core_web_sm; ORG, GPE/LOC, DATE; PhraseMatcher over the vocabulary), extraction.py (RDFLib)	Done
Obj. 3 — star schema + ETL into DuckDB	warehouse.py, etl.py, run_pipeline.py	Done
Obj. 4 — association rules + clustering	mining.py (mlxtend FP-Growth/Apriori; scikit-learn k-Means)	Done
Obj. 5 — Streamlit dashboard with KPIs, filters, rules, clusters + basic evaluation	dashboard/app.py, logic.py; evaluate.py	Done
§4.1 seven-stage pipeline, independently testable stages	one module per stage; tests per stage	Done
§4.2 150–300 public news articles, Indian context, multiple sectors	corpus.py: 240 synthetic articles (default) or any public CSV via --input; four sectors; monsoon, port, policy themes	Done (see note §4.1)
§4.3 tokenisation, stop-words, lemmatisation	nlp.preprocess → clean_text column	Done
§4.3 risk vocabulary grouped by type (weather, policy, supplier, logistics, reputational)	data/risk_vocabulary.json (5 types, weighted terms)	Done
§4.3 co-occurrence rule within paragraph (else article)	extraction.extract_events; split_paragraphs falls back to whole article when no blank-line paragraphs exist	Done
§4.3 manual review of a random triple sample	outputs/triple_review_sample.csv (50 rows, paragraph shown, column for manual Y/N) + automatic precision vs ground truth	Done
§4.4 SPARQL as validation mechanism	semantic.VALIDATION_QUERIES (6 checks)	Done
§4.5 Table 4.1 fact + 3 dimensions; entity & source degenerate	warehouse.DDL	Done
§4.5 quality checks: nulls, duplicates, date parsing, referential checks	etl.clean, warehouse.referential_checks	Done
§4.5 semantic enrichment via shared event identifier	BRIDGE_SEMANTIC_LINK + ork:eventId	Done
§4.6 transactions of risk type, region, sector, severity; support/confidence/lift	mining.build_transactions / mine_rules	Done
§4.6 k-Means on impact, mentions, encoded risk type, temporal features; elbow + silhouette	mining.feature_matrix / cluster_events	Done
§4.7 single page: KPI row, OLAP filter (region, time, risk type), rules table, cluster scatter, caching	dashboard/app.py with st.cache_data / st.cache_resource	Done
Table 4.3 KPIs: Risk Exposure Index, Avg Impact, Events in View, Top Rule Confidence	dashboard/logic.kpis	Done

Report requirement	Implementation	Status
§4.8 tools: pandas, spaCy, RDFLib, Protégé, SPARQL, mlxtend, scikit-learn, DuckDB, Streamlit, Plotly, pytest, GitHub	requirements.txt; branch-per-stage workflow described in §10	Done
Out of scope: decision tree, DBSCAN, rule-based alerts, SVM, BIRCH, OPTICS	deliberately not built (documented as future scope, §12)	Excluded by design

3. Architecture

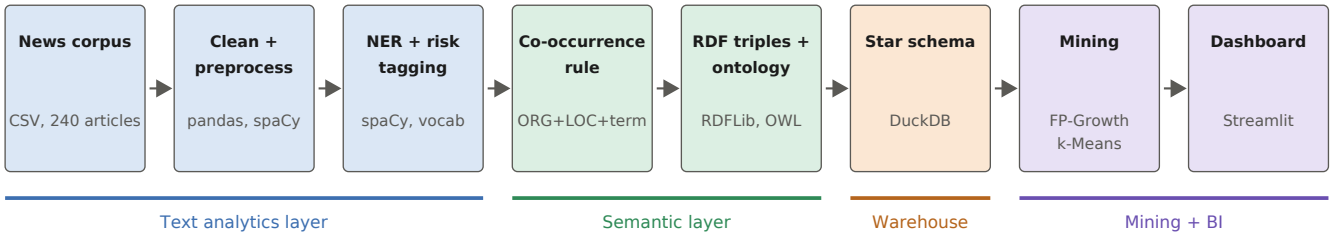


Figure 1 — The implemented OntoRisk BI pipeline (each stage's output is the next stage's input).

The system is a linear, layered pipeline. Each stage reads a file or table written by the previous one, so each can be tested and replaced on its own, and the four team members could build adjacent stages in parallel against agreed interfaces (a cleaned-article DataFrame after Week 1, a populated warehouse after Week 2). `run_pipeline.py` executes all of them in order and prints one log line per stage.

#	Stage	Module	Input → Output
1	Corpus collection	corpus.py	generator / external CSV → data/raw/news_corpus.csv
2	Ontology (T-Box)	ontology.py	vocabulary → ontology/ontorisk.ttl, .owl
3	Cleaning & preprocessing	etl.py, nlp.preprocess	raw CSV → articles_clean.csv + quality report
4	NER + co-occurrence rule	nlp.py, extraction.py	clean articles → risk_events.csv, entities.csv
5	RDF + SPARQL validation	extraction.py, semantic.py	events → ontorisk_kg.ttl, validation JSON
6	Semantic ETL to warehouse	warehouse.py	events → DuckDB star schema + bridge
7	Mining	mining.py	V_RISK_EVENTS → rules, itemsets, clusters, profiles
8	Evaluation + figures	evaluate.py, figures.py	events + gold → metrics.json, PNG figures
—	Dashboard	dashboard/app.py	warehouse + mining outputs + KG → web page

Folder structure

```
ontorisk-bi/
├── run_pipeline.py            # one command runs every stage
├── requirements.txt  README.md  pytest.ini
├── data/
│   ├── risk_vocabulary.json   # curated risk vocabulary (project artefact)
│   ├── location_gazetteer.json
│   ├── raw/         news_corpus.csv, gold_events.csv
│   └── processed/   articles_clean.csv, entities.csv, risk_events.csv
├── ontology/        ontorisk.owl (Protégé), ontorisk.ttl, ontorisk_kg.ttl
├── warehouse/       ontorisk.duckdb, olap_queries.sql
├── src/ontorisk/    config, corpus, nlp, extraction, ontology, semantic,
│                    etl, warehouse, mining, evaluate, figures
├── dashboard/       app.py (Streamlit), logic.py (tested data layer)
├── tests/           test_pipeline.py, test_dashboard_smoke.py
├── outputs/         metrics.json, association_rules.csv, clusters.csv, figures/ ...
└── docs/            this explanation PDF and its generator
```

4. How each stage works

4.1 Data collection — the corpus

The report plans 150–300 public news articles (Kaggle and other public sources) with Indian regional context and coverage of four sectors. The repository ships a **curated synthetic corpus of 240 articles** generated by `corpus.py`, and the same pipeline runs unchanged on any real news CSV (`python run_pipeline.py --input news.csv`; column names such as headline/content/published_at are mapped automatically).

Why a synthetic corpus? (1) It is reproducible (fixed random seed), so every team member and the examiner get identical results. (2) Because the generator knows which company, place and risk each story is really about, it writes a ground-truth file (`gold_events.csv`), which turns the planned manual precision check into an exact, automatic measurement over all triples. (3) It avoids copyright and scraping issues. Companies are fictional (e.g. Konkan Freight Ltd, Kalinga Steel Works); places and seasonality are real Indian context.

- **Five risk types** with realistic biases: weather (Mumbai, Chennai, Kochi, Guwahati, Odisha; 75% in June–September), logistics (Nhava Sheva, Mundra, Chennai, Kolkata, Visakhapatnam, Colombo, Singapore), policy (mostly New Delhi), supplier (Hosur, Pune, Bengaluru, Coimbatore, Jamshedpur), reputational (Bengaluru, Gurugram, Hyderabad, Noida).
- **Co-occurring risks**: 55% of weather stories also report port/freight disruption; some policy stories lead to supplier shortages — patterns the miner should rediscover.
- **Realistic noise**: 65% of articles contain an analyst/industry paragraph that names another organisation and city, sometimes with a risk word — exactly the incidental co-mention that produces spurious triples.
- **Dirty records** for the ETL to catch: one duplicate, one unparseable date, one empty article (rejected: 1 duplicate id, 0 duplicate text, 1 bad date, 1 empty).

Example article

```
A0001 | 2024-12-22 | Synthetic Regional Bulletin
TITLE: Vijaynagar Credit Ltd reviews business after GST revision

Officials in New Delhi confirmed a GST revision that Vijaynagar Credit Ltd said would raise compliance
costs across its bank operations. The company expects delays of 3 to 11 days and said it was working to
restore normal schedules.

Ashoka Analytics, based in Ahmedabad, noted that broader credit demand remained steady despite talk of
social media backlash.

The company said it would update stakeholders as the situation evolves.
```

4.2 Preprocessing and named entity recognition

Backend used in this run: spaCy 3.8.7 / en_core_web_sm 3.8.0.

Each article is split into paragraphs (blank-line separated). Two things happen with spaCy:

- **Preprocessing** (`nlp.preprocess`): the spaCy pipeline tokenises the text; stop words, punctuation, spaces and numbers are dropped and every remaining token is replaced by its lower-case *lemma* (“disrupted” → “disrupt”, “floods” → “flood”). The result is stored in `clean_text` for analysis.
- **NER** runs on the *original* paragraph (NER needs capital letters and function words). The pre-trained `en_core_web_sm` model labels ORG (organisations), GPE/LOC (places) and DATE spans.
- **Normalisation**: organisation names lose possessives and leading words (“Shares of …”); places are looked up in `location_gazetteer.json` to get region/state/country. A gazetteer pass also recovers multi-word Indian place names the small model may miss (e.g. “Nhava Sheva”) — standard NER + knowledge-base linking practice. Unmappable places are ignored.

- **Risk tagging**: spaCy's `PhraseMatcher` (on lower-cased text) finds every vocabulary term; `filter_spans` keeps the longest match so “port congestion” is counted once, not also as “congestion”.

Entity label	Mentions found
LOC	503
RISK_TERM	432
ORG	399
DATE	256
Distinct organisations	32
Distinct places	39

Risk vocabulary (data/risk_vocabulary.json)

Risk type	Sub-type	Example terms (weight 1–3)
weather	Monsoon / natural disaster	monsoon (2), heavy rainfall (2), heavy rain (2), flood (3), floods (3), flooding (3), waterlogging (2), cyclone (3) …
logistics	Port congestion / transport disruption	port congestion (3), congestion (2), container shortage (2), vessel backlog (2), berthing delays (2), freight delays (2), truckers' strike (3), transporters' strike (3) …
policy	Regulatory / policy change	export ban (3), import duty (2), tariff hike (2), gst revision (2), gst (1), regulatory crackdown (3), rbi circular (2), licence suspension (3) …
supplier	Supplier failure / input shortage	supplier insolvency (3), raw material shortage (2), plant shutdown (3), chip shortage (2), supplier default (3), labour strike (2), production halt (3), input shortage (2) …
reputational	Brand / reputational incident	product recall (3), recall (2), consumer boycott (2), data breach (3), fraud allegation (3), fraud allegations (3), social media backlash (2), quality complaints (2) …

The vocabulary also holds **sector terms** (freight, cargo → Supply Chain & Logistics; stores, e-commerce → Retail; bank, loan → Finance; plant, factory → Manufacturing) used to assign a sector to every event from the organisation name and its paragraph, and **intensity terms** (severe, halted, crore, stranded … vs. minor, resumed, limited impact) used in the impact score.

4.3 The co-occurrence relation rule

Rule. In every paragraph that contains at least one ORGANISATION, one LOCATION and one RISK TERM, create a RiskEvent for each (organisation, location, risk type) combination and emit `<entity> ork:affects <event>` and `<event> ork:locatedIn <location>` plus data-property triples (category, impact score, severity, date, provenance). Nothing is learned — the rule is fast, explainable and auditable; its known cost is precision.

Two clean-ups follow: if an event is found both at a city and at that city's own state (“Mumbai, Maharashtra”) only the more specific city is kept, and duplicate (article, organisation, place, risk type) combinations are merged. Each event receives a deterministic id (EV_ + MD5 of article|org|place|type), shared by its RDF node and its fact row.

Impact score and severity (transparent heuristic)

```
impact = 0.5 + 1.5 x (highest vocabulary weight of matched terms, 1..3)
       + 0.8 x min(mention_count - 1, 2)        # repeated coverage of the same risk type
       + 1.6 x article_intensity                 # +severe/halted/crore..., -minor/resumed...
       + 1.0 if the event is in the lead paragraph
impact clipped to [0, 10];   severity: Low < 4 <= Medium < 7 <= High
```

In this run: 143 High, 168 Medium and 76 Low events; mean impact 6.16.

Worked example — article A0039

```
[para 0] Hosur Auto Components Ltd warned investors that the new compliance norms unveiled in Mumbai could
squeeze manufacturing margins. Executives said deliveries were partially suspended and margins could
come under some pressure.

[para 1] Hosur Auto Components Ltd added that the new compliance norms could lead to a chip shortage for
units in Nashik.

[para 2] Market watchers at Lotus Ratings Agency said investors in Hyderabad were largely unmoved by the
news.

[para 3] Shares of the company were little changed in afternoon trade.
```

Event	Para.	Entity —affects→	—locatedIn→ place	Type	Impact	Severity
EV_b3b1733ef0	0	Hosur Auto Components Ltd	Mumbai	policy	5.78	Medium
EV_7bf008ba31	0	Hosur Auto Components Ltd	Hosur	policy	5.78	Medium
EV_94a1e75bf7	1	Hosur Auto Components Ltd	Nashik	supplier	3.98	Low
EV_7d4d8f36e9	1	Hosur Auto Components Ltd	Hosur	supplier	3.98	Low
EV_12a0ccf2d0	1	Hosur Auto Components Ltd	Nashik	policy	4.78	Medium

Events from the lead paragraph are the true story; events created from a later paragraph may be genuine (a second disruption) or spurious (an analyst quoted in another city) — the rule cannot tell, which is exactly why precision is measured in §4.8.

4.4 The ontology

Built as code with RDFLib (so it is version-controlled and regenerated on every run) and saved as `ontology/ontorisk.owl` (RDF/XML) which opens directly in **Protégé**, and as Turtle. The design follows §4.4 exactly: six classes, only two object properties, impact and provenance as data properties, no causal relations (the co-occurrence rule cannot distinguish cause from association).

Class	Meaning
RiskEvent	A business-risk occurrence detected in text, e.g. a flood disrupting a warehouse.
Entity	An organisation (company, bank, retailer...) that a risk event affects.
Location	A geographic place (city, port, state, country) where a risk event occurs.
RiskType	The category of a risk: weather, logistics, policy, supplier or reputational.
SourceArticle	The news article a risk event was extracted from (provenance).
RiskTerm	A controlled-vocabulary keyword that signals a risk type (e.g. 'port congestion').

Object property	Domain → Range	Meaning
affects	Entity → RiskEvent	The entity is affected by (exposed to) the risk event.
locatedIn	RiskEvent → Location	The risk event takes place in the location.

Data properties (selection)	Domain	Type
eventId	RiskEvent	string
impactScore	RiskEvent	decimal
severityBand	RiskEvent	string
riskCategory	RiskEvent	string
mentionCount	RiskEvent	integer
eventDate	RiskEvent	date

Data properties (selection)	Domain	Type
sourceId	RiskEvent	string
matchedTerm	RiskEvent	string
sector	Entity	string
entityName	Entity	string
placeName	Location	string
region	Location	string

Extra axioms: the six classes are pairwise `owl:disjointWith`; `locatedIn`, `eventId`, `impactScore`, `severityBand` and `riskCategory` are functional. The five RiskType individuals and every vocabulary term (as a RiskTerm individual with its weight) are part of the ontology, so the vocabulary itself is machine-readable.

What the generated triples look like (Turtle)

```
@prefix ork: <http://ontorisk.example.org/ontology#> .
@prefix ord: <http://ontorisk.example.org/data/> .

ord:entity/Vijaynagar_Credit_Ltd  a ork:Entity ;
    ork:entityName "Vijaynagar Credit Ltd" ; ork:sector "Finance" ;
    ork:affects ord:EV_686ccd1aba .

ord:EV_686ccd1aba  a ork:RiskEvent ;
    ork:locatedIn ord:location/New_Delhi ;
    ork:riskCategory "policy" ; ork:impactScore 4.5 ;
    ork:severityBand "Medium" ; ork:eventDate "2024-12-22"^^xsd:date ;
    ork:sourceId "A0001" ; ork:matchedTerm "gst revision" .
```

Opening it in Protégé

- Install Protégé 5.6+ (protege.stanford.edu) → File → Open → `ontology/ontorisk.owl`.
- Entities tab: browse the 6 classes, 2 object properties, data properties and the RiskType / RiskTerm individuals. OntoGraf tab: visual graph.
- Reasoner → HermiT → Start reasoner: checks the ontology is consistent.
- To inspect the populated graph, open `ontology/ontorisk_kg.ttl` instead (larger).

4.5 SPARQL validation

After triples are generated the knowledge graph is checked with SPARQL (RDFLib's built-in engine). All checks must pass; they also run in the test suite.

Validation query	Passed	Result
ontology has six classes	yes	6
exactly two object properties	yes	2
every event has a location	yes	0
every event affects an entity	yes	0
domain range respected	yes	0
events per risk type	yes	logistics 113; weather 86; policy 74; supplier 64; reputational 50

```
# Example: every High-severity event in Maharashtra, with the affected entity
SELECT ?entity ?type ?place ?date WHERE {
  ?e ork:severityBand "High" ; ork:riskCategory ?type ; ork:eventDate ?date ;
     ork:locatedIn ?loc .
  ?loc ork:state "Maharashtra" ; ork:placeName ?place .
  ?x ork:affects ?e ; ork:entityName ?entity } ORDER BY ?date
```

4.6 The warehouse (DuckDB star schema)

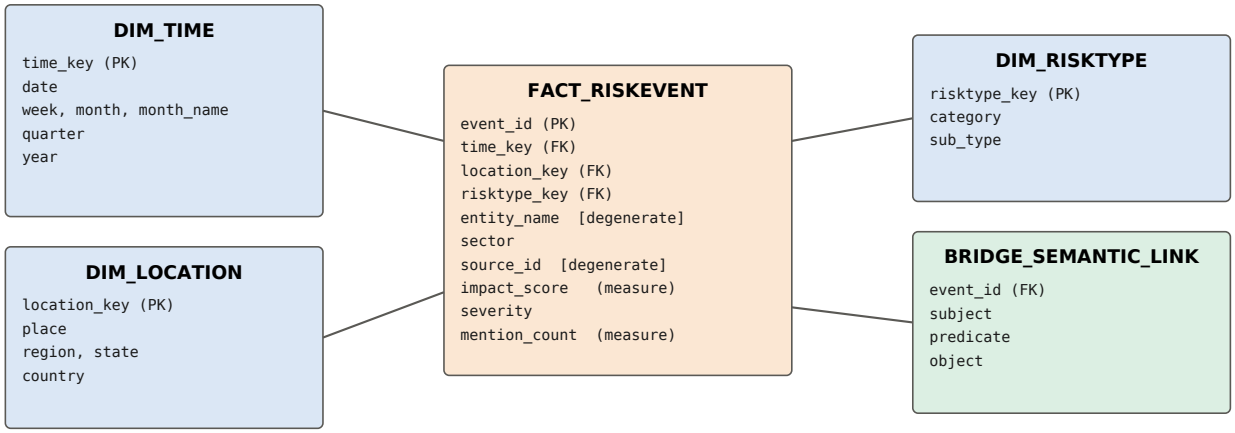


Figure 2 — Implemented star schema (orange = fact, blue = dimensions) and the semantic bridge table (green) that links each fact row to its RDF triples.

Engine used in this run: SQLite (fallback - DuckDB not installed). The schema follows Table 4.1. Entity and source are **degenerate attributes** on the fact row (Kimball & Ross): they are never browsed hierarchically, so separate dimension tables would only add joins. DIM_LOCATION also stores the `place` (city/port) so the dashboard can drill down Region → State → Place, and DIM_TIME stores `month_name` for display.

Table	Rows
DIM_TIME	183
DIM_LOCATION	32
DIM_RISKTYPE	5
FACT_RISKEVENT	387
BRIDGE_SEMANTIC_LINK	1935
STG_ARTICLES	240

The ETL (pandas) loads dimensions first, then facts, then the bridge. Afterwards the **referential checks** run: fact rows = 387, orphan time = 0, orphan location = 0, orphan risktype = 0, facts without triples = 0, null measures = 0.

Hybrid (semantic) link. Table `BRIDGE_SEMANTIC_LINK` stores, for every event_id, the RDF triples generated for it, and every RDF event carries `ork:eventId`. So SQL can jump from a fact row to its triples, and a SPARQL result can jump back to the fact row. A view, `V_RISK_EVENTS`, joins the fact with all dimensions and feeds the miners and the dashboard. `warehouse/olap_queries.sql` shows ROLLUP, drill-down, CUBE, slice/dice and the hybrid join.

```
-- Roll-up with subtotals (DuckDB)
SELECT region, risk_type, COUNT(*) AS events, ROUND(AVG(impact_score),2) AS avg_impact
FROM V_RISK_EVENTS GROUP BY ROLLUP (region, risk_type)
ORDER BY region NULLS LAST, events DESC;
```

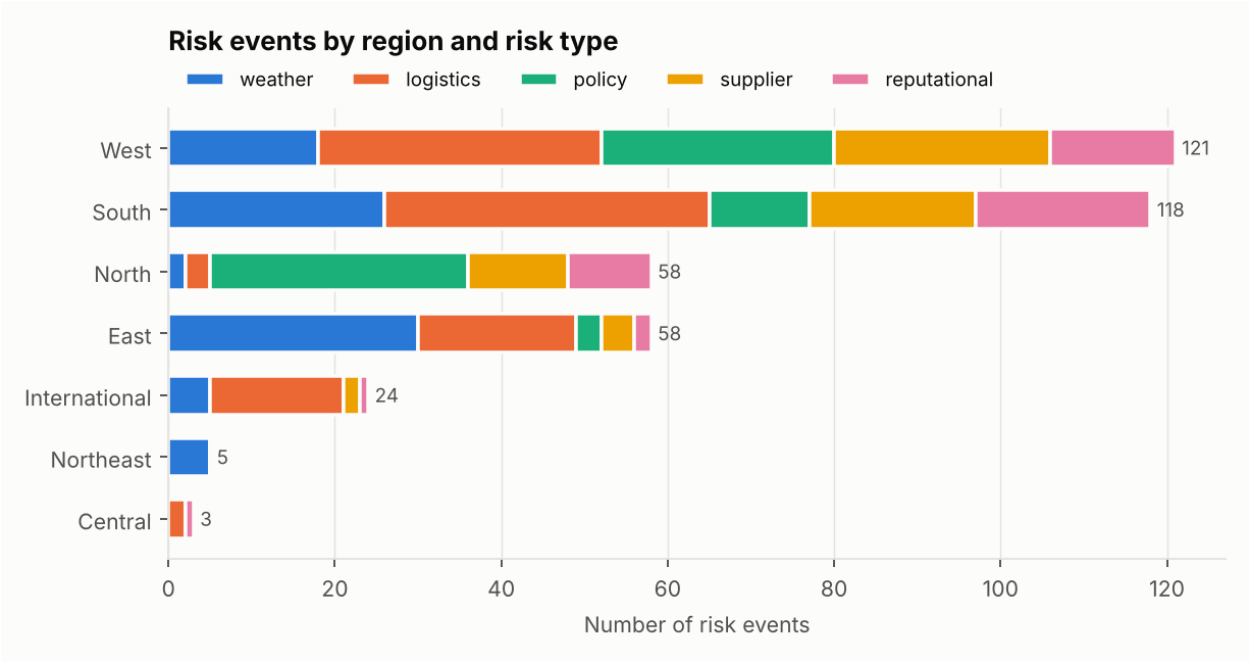


Figure 3 — Risk events by region and type (from V_RISK_EVENTS).

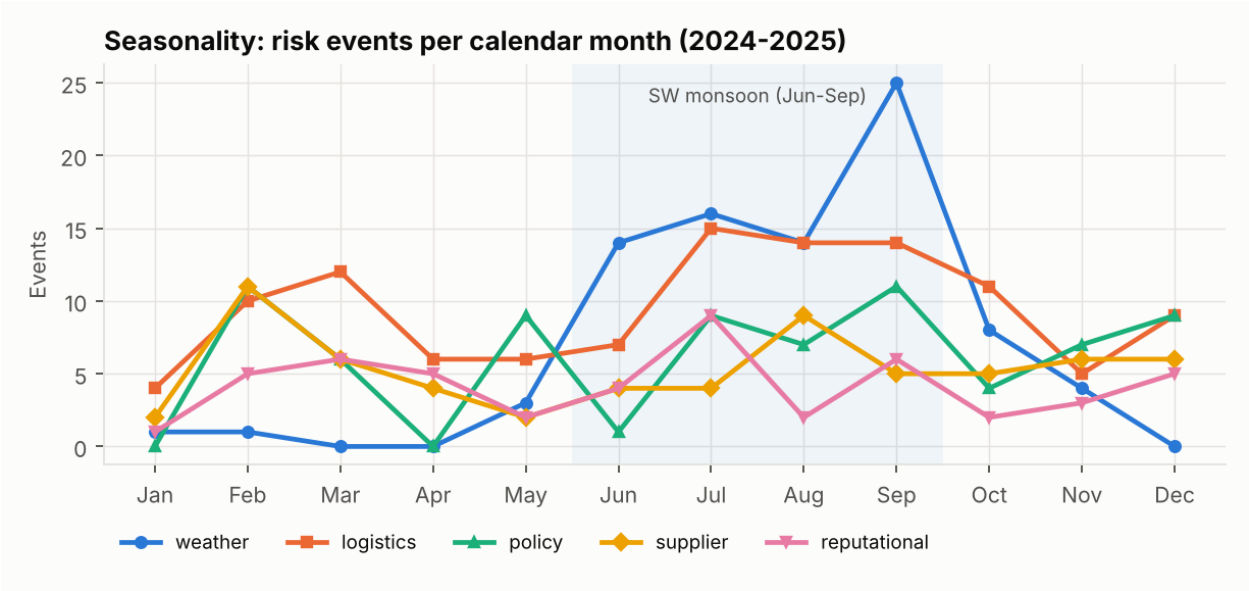


Figure 4 — Seasonality: weather events peak in the June–September monsoon.

4.7 Data mining

Association rules (FP-Growth, Apriori as cross-check)

Each risk event becomes a **transaction** — a basket of items describing it:

```
{ risktype=weather, region=West, sector=Supply Chain & Logistics, severity=High, co_risk=logistics }
```

The four items from §4.6 (risk type, region, sector, severity band) are extended with `co_risk=<type>` for every *other* risk type reported in the same article — this is what lets the miner find co-occurring risk factors such as weather together with logistics. FP-Growth (mlxtend) finds itemsets with support ≥ 5%; rules are kept when confidence ≥ 60% and lift ≥ 1.1, with a single-item consequent for readability. Apriori run on the same data returns the identical rule set: **yes**.

If (antecedent)	Then (consequent)	Support	Confidence	Lift
co_risk=logistics, sector=Supply Chain & Logistics, severity=High	risktype=weather	0.057	0.88	3.96

If (antecedent)	Then (consequent)	Support	Confidence	Lift
sector=Finance, severity=Medium	risktype=policy	0.059	0.66	3.44
co_risk=logistics, sector=Supply Chain & Logistics	risktype=weather	0.080	0.76	3.40
co_risk=logistics, severity=High	risktype=weather	0.096	0.74	3.33
risktype=policy, severity=Medium	co_risk=supplier	0.067	0.60	3.30
risktype=weather, sector=Supply Chain & Logistics, severity=High	co_risk=logistics	0.057	0.76	3.23
co_risk=supplier, severity=Medium	risktype=policy	0.067	0.60	3.16
risktype=weather, sector=Supply Chain & Logistics	co_risk=logistics	0.080	0.70	3.00
risktype=weather, severity=High	co_risk=logistics	0.096	0.70	2.97
region=East, severity=High	risktype=weather	0.052	0.65	2.90
risktype=weather	co_risk=logistics	0.140	0.63	2.67
sector=Supply Chain & Logistics, severity=Medium	risktype=logistics	0.075	0.66	2.26

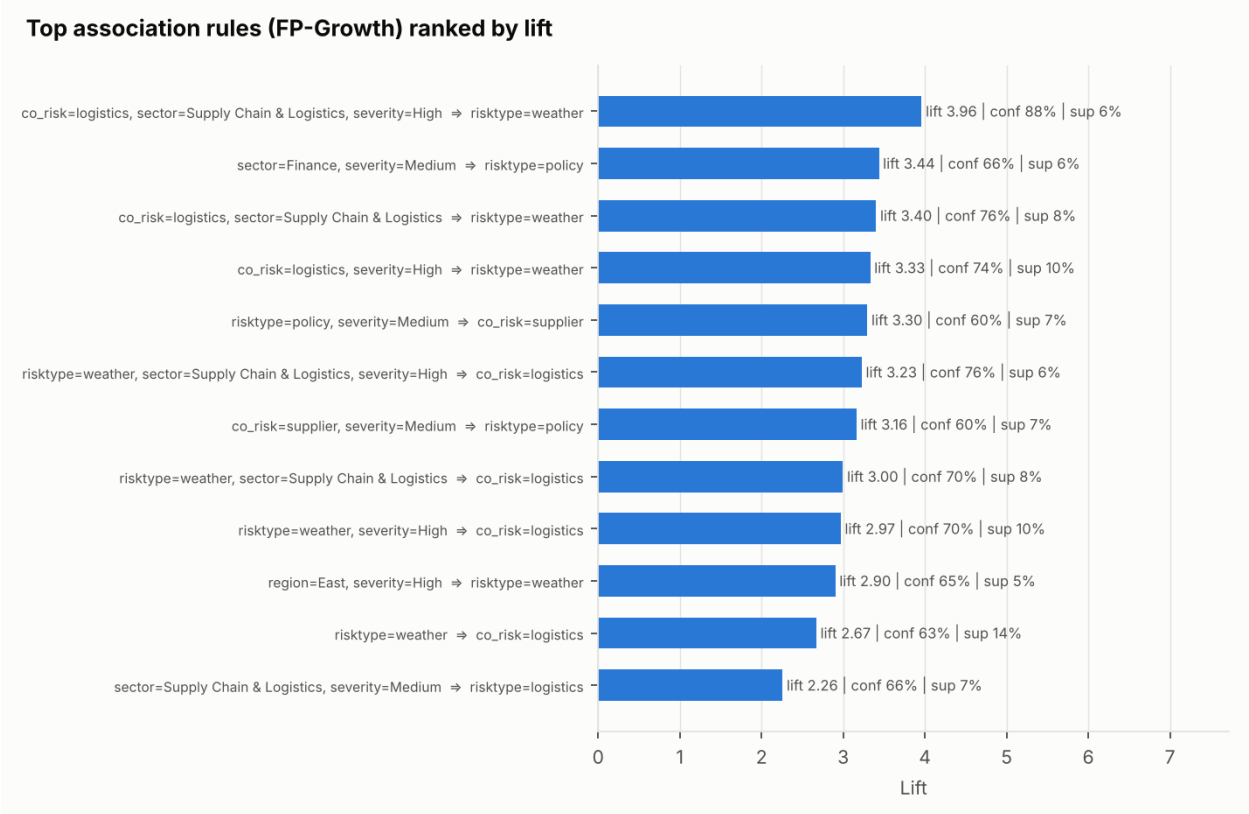


Figure 5 — Strongest rules by lift.

How to read the top rule. “co_risk=logistics, sector=Supply Chain & Logistics, severity=High ⇒ risktype=weather”: 5.7% of all events contain both sides (support); when the left side holds, the right side holds 88% of the time (confidence); and that is 3.96× more often than if the two were independent (lift). The rules rediscover the monsoon story: weather disruption co-occurs with port/freight disruption, hits logistics and manufacturing hardest and tends to be High severity.

k-Means clustering

Feature vector per event: impact score, mention count, one-hot risk type (5 columns), and temporal features — month as sin/cos (so December sits next to January) and a monsoon-season flag. All features are standardised. Because the five one-hot columns describe one attribute (and the three

time columns another), each block is divided by √(block size) so it weighs as much as a single numeric feature — otherwise the risk-type columns alone dictate the clusters.

k is scanned over 2–10; the **elbow** is detected automatically (the point of the inertia curve furthest from the straight line joining its ends) and validated with the **silhouette score**. Elbow k = 5, best-silhouette k = 2, chosen k = 5 (silhouette 0.239). PCA projects the vectors to 2-D for the scatter plot (PC1+PC2 explain 56% of variance).

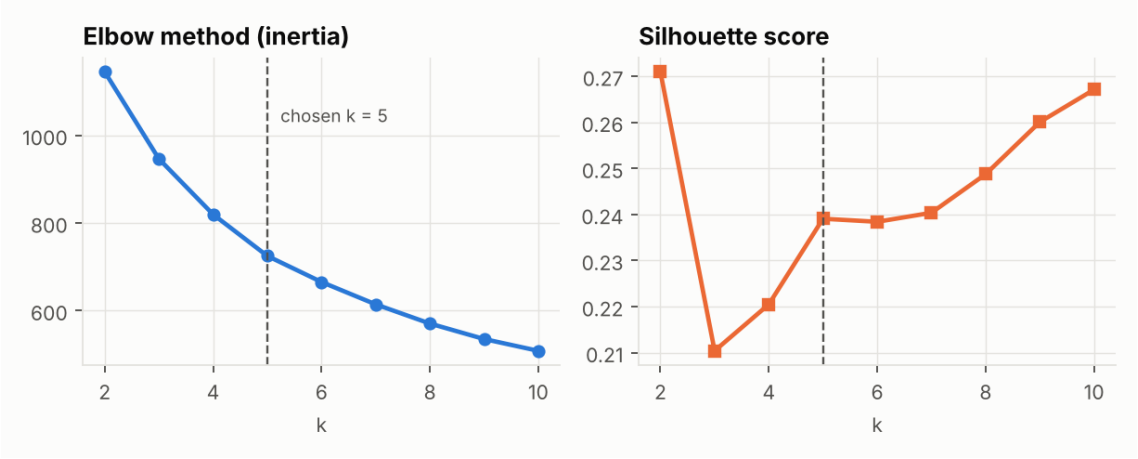


Figure 6 — Elbow and silhouette diagnostics.

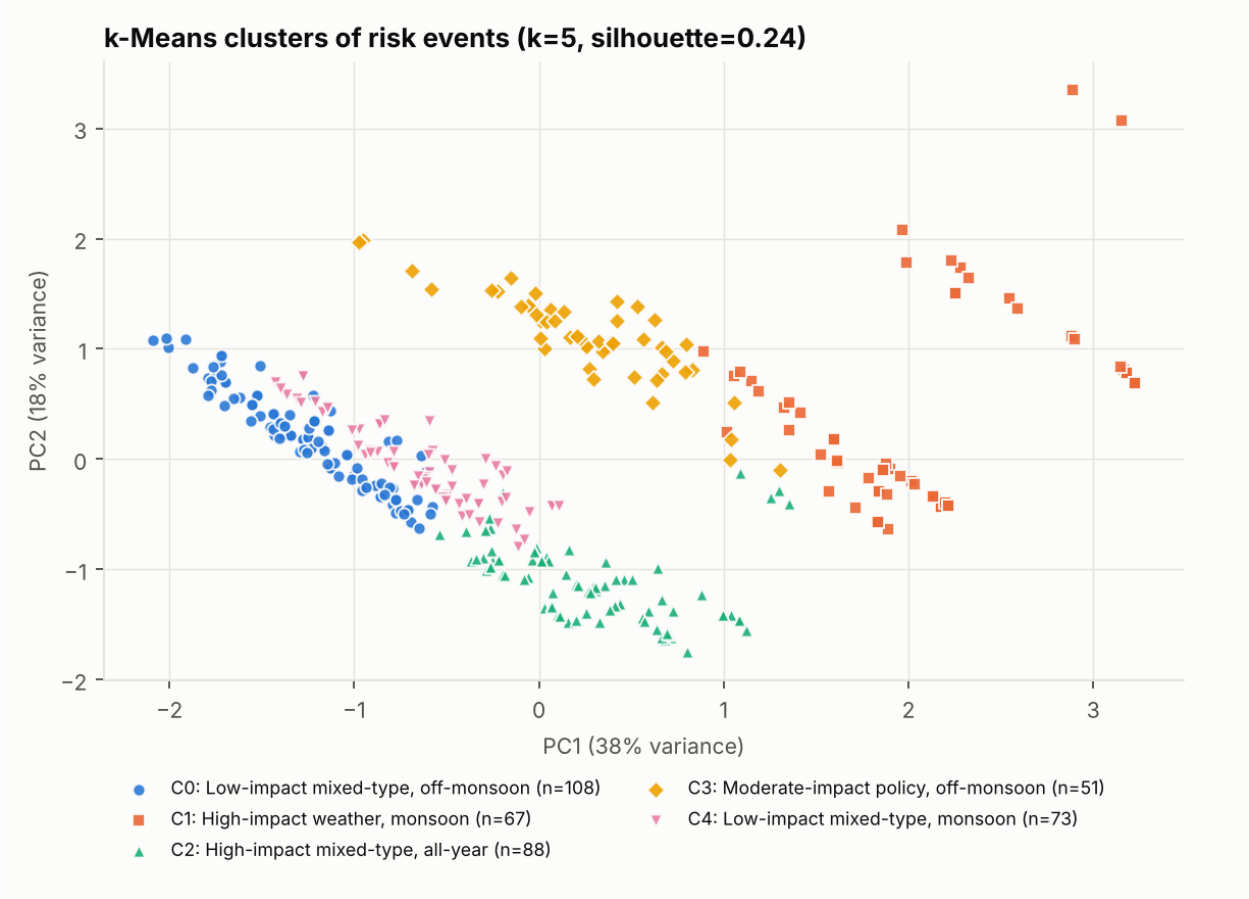


Figure 7 — Risk events coloured by cluster (PCA projection).

Cluster	Label	Size	Avg impact	Top type (share)	Monsoon share	High share
C0	Low-impact mixed-type, off-monsoon	108	4.32	logistics (36%)	0%	1%
C1	High-impact weather, monsoon	67	8.57	weather (84%)	90%	78%

Cluster	Label	Size	Avg impact	Top type (share)	Monsoon share	High share
C2	High-impact mixed-type, all-year	88	8.56	logistics (46%)	51%	97%
C3	Moderate-impact policy, off-monsoon	51	5.18	policy (61%)	24%	6%
C4	Low-impact mixed-type, monsoon	73	4.44	reputational (26%)	100%	3%

A silhouette around 0.2–0.3 means the groups are real but overlap — normal for mixed categorical/numeric event data; the clusters are still clearly interpretable (e.g. a high-impact monsoon-weather group versus low-impact off-monsoon events).

4.8 Evaluation of the extraction layer

Against the generator's ground truth, the co-occurrence rule produced 387 events, of which 280 are correct: **precision 72.4%**. It found 275 of the 297 true events: **recall 92.6%** (F1 0.812). On the 50-triple review sample the precision is 70.0%.

Risk type	Precision
logistics	83.2%
policy	62.2%
reputational	70.0%
supplier	82.8%
weather	60.5%

Error analysis. Almost all errors are the incidental co-mentions the report predicted: an analyst firm quoted in another city next to a risk word, or a second paragraph where a weather word and a port appear together ("the floods also triggered congestion at Mundra") so the rule also creates a *weather*@Mundra event. Weather has the lowest precision for that reason. Recall is high because the rule is deliberately permissive; the events it misses come from the NER step — organisations the small spaCy model did not tag as ORG (e.g. some retail names) or span-boundary errors ("Northstar Securities in Hyderabad", partly repaired by the name normaliser). Real news will give lower recall (more varied phrasing) — the manual review sheet `outputs/triple_review_sample.csv` is provided for exactly that case.

Why the paragraph window matters. As an ablation, the first 160 articles were re-run with their paragraph breaks removed (so the rule falls back to the whole article, as it would for a CSV that stores each article as one block). The same text then produced 793 events instead of 256 — about three times as many, almost all of the extra ones spurious. Keeping paragraph boundaries when collecting real news is therefore the single most useful thing for precision.

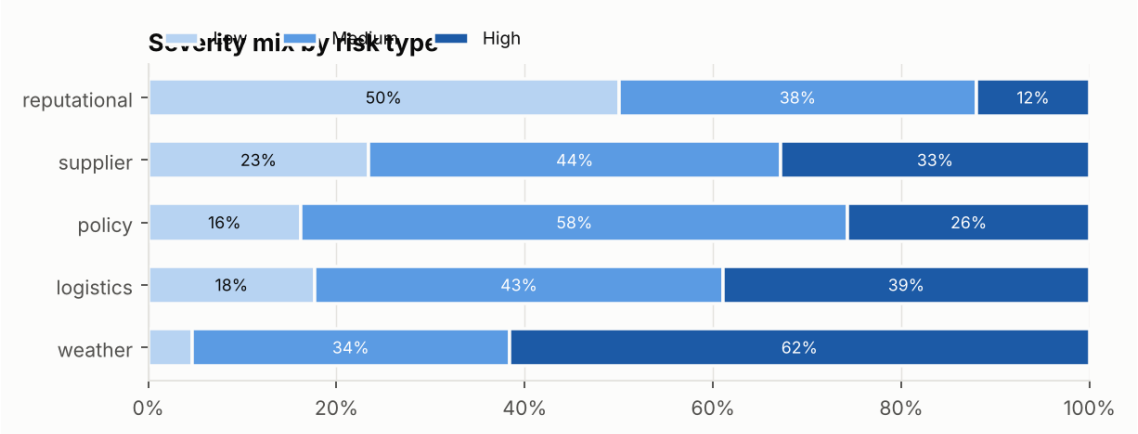


Figure 8 — Severity mix by risk type.

4.9 The dashboard

A single Streamlit page (layout of Figure 4.4 in the report), connected directly to the DuckDB warehouse and the mining outputs. Top to bottom:

- **KPI cards** (Table 4.3): Risk Exposure Index = Σ impact × severity weight (Low 1, Medium 2, High 3) with its share of total exposure; Average Impact Score; Events in View (of total); Top Rule Confidence (confidence of the highest-lift rule shown).
- **OLAP filters**: Region (multi-select), Time period (date range), Risk type (multi-select). A roll-up / drill-down panel re-aggregates the selection at Region → State → Place and Year → Quarter → Month for a chosen measure (events, average impact, exposure). Rule thresholds can be tuned with sliders.
- **Association rules table**: FP-Growth is *re-run on the filtered events*, so the rules respond to the selection (minimum 25 events).
- **k-Means scatter**: events in view, coloured and shaped by cluster, with hover details, plus a cluster profile table.
- Extras: a SPARQL console over the knowledge graph (example queries included) and the event table.
- Performance: data loading and rule mining are cached with `st.cache_data`; the RDF graph with `st.cache_resource`.

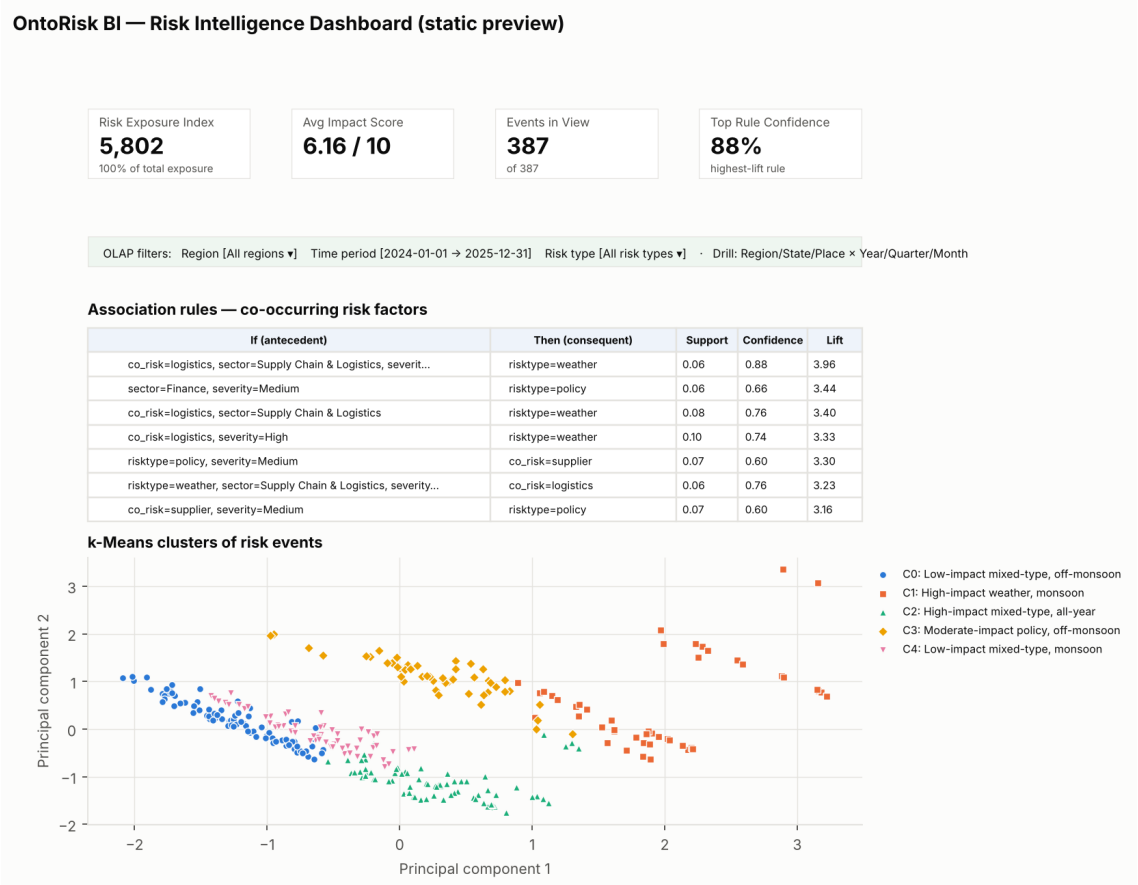


Figure 9 — Static rendering of the dashboard's content with no filter applied (the live Streamlit page is interactive: hover, filters, drill-down, SPARQL console).

5. Technologies — what, how it is used here, why it matters

Python 3

What it is. A general-purpose, interpreted language with the richest ecosystem for data science, NLP and the semantic web.

How OntoRisk BI uses it. Every stage, from corpus generation to the dashboard, is Python; the package lives in `src/ontorisk` and one script runs the whole pipeline.

Significance in the project. One language for NLP, RDF, SQL, mining and UI means no glue code between tools and an easy hand-off between team members.

Good to know. Type hints, dataclasses (`Paragraph`, `GoldEvent`) and a central `config.py` keep the code readable. Python 3.10–3.13 is supported.

pandas

What it is. The standard Python library for tabular data: the DataFrame (labelled, typed columns) with fast filtering, grouping, joining and I/O.

How OntoRisk BI uses it. Reads the corpus CSV, performs all cleaning and quality checks (duplicates, empty text, date parsing), builds the dimension and fact tables, and is the exchange format between stages and with DuckDB.

Significance in the project. It is the 'T' of the ETL. DuckDB can query a pandas DataFrame directly, which removes a separate loading tool.

Good to know. `pd.to_datetime(errors='coerce')` turns bad dates into NaT so they can be counted and rejected; `duplicated()` detects duplicates; `crosstab`/`groupby` produce the OLAP-style summaries.

spaCy (en_core_web_sm)

What it is. An industrial-strength NLP library by Explosion AI (Honnibal & Montani). Its pre-trained pipelines perform tokenisation, part-of-speech tagging, dependency parsing, lemmatisation and named entity recognition. `en_core_web_sm` is the small English pipeline (CNN-based, trained on OntoNotes 5; 18 entity types).

How OntoRisk BI uses it. Tokenisation, stop-word removal and lemmatisation in preprocessing; NER for ORG, GPE, LOC and DATE; the `PhraseMatcher` (with `attr='LOWER'`) for case-insensitive vocabulary matching; `filter_spans` for longest-match.

Significance in the project. NER is how unstructured text becomes candidate subjects and objects for RDF triples. Using a pre-trained model needs no annotated training data, which is what makes the project feasible in three weeks (Chapter 2 of the report).

Good to know. Larger models (`en_core_web_md/lg/trf`) are more accurate but slower; fine-tuning on Indian business news, or an EntityRuler with company lists, would raise recall. If spaCy is unavailable the code falls back to a regex/gazetteer recogniser so it still runs.

RDF and RDFLib

What it is. RDF (Resource Description Framework, W3C) represents knowledge as *triples* — subject, predicate, object — each identified by an IRI, forming a graph. RDFLib is the main Python library to create, parse, serialise (Turtle, RDF/XML, N-Triples, JSON-LD) and query RDF graphs.

How OntoRisk BI uses it. The ontology (T-Box) and the generated facts (A-Box) are built as an RDFLib `Graph` under the namespaces `ork:` (ontology) and `ord:` (data), and saved as Turtle / RDF/XML.

RDFLib's SPARQL engine runs the validation and dashboard queries.

Significance in the project. RDF is the semantic layer: it gives each extracted fact explicit meaning (types and properties defined in the ontology) and lets heterogeneous facts be linked and queried by relationship rather than by table layout.

Good to know. Turtle is the human-readable syntax (`ord:x ork:affects ord:y .`). Typed literals (xsd:decimal, xsd:date) keep values comparable. Large graphs would move to a triple store such as Apache Jena Fuseki or GraphDB.

OWL ontology and Protégé

What it is. OWL (Web Ontology Language) adds vocabulary on top of RDF to define classes, properties, domains, ranges, disjointness and cardinality, so a reasoner can check consistency and infer facts. Protégé (Stanford, Musen) is the most widely used open-source ontology editor.

How OntoRisk BI uses it. Six owl:Class, two owl:ObjectProperty with domain/range, typed owl:DatatypeProperty, disjoint classes and functional properties, plus RiskType and RiskTerm individuals. The file `ontorisk.owl` is designed to be opened, inspected, visualised (OntoGraf) and reasoned over (HermiT) in Protégé.

Significance in the project. The ontology is the shared schema of the semantic layer — it states what a RiskEvent is and how it relates to an Entity and a Location — and the SPARQL checks verify the extracted data actually conforms to it.

Good to know. Keeping the ontology small is a design decision from the literature: a limited class/property set is easier to populate consistently from automatic extraction. Causal properties (causes, mitigates) were deliberately not defined because co-occurrence cannot populate them faithfully.

SPARQL

What it is. The W3C query language for RDF (SPARQL 1.1). Queries match graph patterns of triples with variables and support FILTER, OPTIONAL, aggregation (GROUP BY, COUNT, AVG), sub-queries and NOT EXISTS.

How OntoRisk BI uses it. Six validation queries (class count, property count, every event has a location and an affected entity, domain/range respected, events per type) and four example semantic queries exposed in the dashboard.

Significance in the project. It is the planned validation mechanism (§4.4) and gives associative, relationship-first queries — e.g. “entities exposed to more than one risk type” — that are awkward in SQL.

Good to know. `FILTER NOT EXISTS` is used to find violations (an event without a location). SPARQLWrapper would be used instead of RDFLib to query a remote endpoint.

DuckDB

What it is. An embedded, in-process analytical (OLAP) database — “SQLite for analytics”: column-oriented, vectorised execution, no server, a single file, full SQL including ROLLUP/CUBE/GROUPING SETS and window functions, and zero-copy access to pandas DataFrames.

How OntoRisk BI uses it. Stores the star schema (FACT_RISKEVENT, DIM_TIME, DIM_LOCATION, DIM_RISKTYPE), the semantic bridge, the staging table and the denormalised view V_RISK_EVENTS. Data are loaded straight from DataFrames (`register` + `INSERT … SELECT`); the dashboard opens it read-only.

Significance in the project. It provides warehouse behaviour (keys, joins, fast aggregation) on a laptop without installing a database server — exactly the reason given in §4.5.

Good to know. If DuckDB is not installed the same schema is created in SQLite so the pipeline still runs; `metrics.json` records the engine. The DuckDB CLI can run `olap_queries.sql` directly.

Star schema, Kimball and Inmon

What it is. Dimensional modelling (Kimball & Ross) organises a warehouse into a central fact table of numeric measures at a declared grain, surrounded by descriptive dimension tables. Inmon's view stresses an integrated, subject-oriented warehouse fed from many sources.

How OntoRisk BI uses it. Grain = one detected risk event. Measures: impact_score, mention_count (severity as a band). Dimensions: time, location (with a Region → State → Place hierarchy), risk type. Degenerate attributes: entity name and source id. Surrogate integer keys; time_key = YYYYMMDD.

Significance in the project. The star schema makes OLAP questions (by region, quarter, risk type) simple, fast joins, and it is the relational half of the hybrid warehouse; the RDF graph is the semantic half (Inmon's integration of two representations of the same events).

Good to know. Larger versions would normalise entity and source into dimensions with slowly changing dimension (SCD Type 2) handling.

OLAP operations

What it is. Online Analytical Processing: interactive analysis of multidimensional data. Core operations: roll-up (aggregate to a coarser level), drill-down (finer level), slice (fix one dimension), dice (sub-cube on several), pivot.

How OntoRisk BI uses it. The dashboard's filters slice and dice by region, period and risk type; the roll-up/drill-down panel moves along Region → State → Place and Year → Quarter → Month; olap_queries.sql shows SQL ROLLUP and CUBE.

Significance in the project. OLAP is the main interaction model planned for the dashboard (§2.4).

Good to know. GROUP BY ROLLUP(a, b) returns totals for (a, b), (a) and the grand total in one query.

mlxtend — FP-Growth and Apriori

What it is. mlxtend (Raschka) provides frequent-pattern mining. Apriori (Agrawal & Srikant, 1994) generates candidate itemsets level by level and prunes with the ‘every subset of a frequent itemset is frequent’ property. FP-Growth (Han, Pei & Yin, 2000) compresses the transactions into an FP-tree and mines it recursively without candidate generation — faster on larger data.

How OntoRisk BI uses it. `TransactionEncoder` one-hot encodes the event baskets, `fpgrowth` finds frequent itemsets, `association_rules` derives rules with support, confidence, lift, leverage and conviction; Apriori is run as a cross-check.

Significance in the project. It discovers co-occurring risk factors — the first analytical output of the dashboard — with no labelled data, and the results are directly interpretable.

Good to know. Support(A⇒B) = P(A∧B); Confidence = P(B|A); Lift = Confidence / P(B) (> 1 means positive association, 1 independence); Leverage = P(A∧B) − P(A)P(B); Conviction = (1 − P(B)) / (1 − Confidence).

scikit-learn — k-Means, silhouette, PCA

What it is. scikit-learn is the standard Python machine-learning library. k-Means (MacQueen, 1967) partitions points into k clusters by alternately assigning each point to its nearest centroid and moving each centroid to its cluster mean, minimising within-cluster sum of squares (inertia). The silhouette score (−1…1) compares each point's cohesion with its separation from the next cluster. PCA projects data onto the directions of largest variance.

How OntoRisk BI uses it. `StandardScaler` for scaling, `KMeans(n_init=10, random_state=42)` for k = 2…10, `silhouette_score` for validation, `PCA(2)` for the scatter plot.

Significance in the project. Clustering groups similar disruption events (the second analytical output) and needs only k, which the elbow + silhouette procedure selects.

Good to know. k-Means assumes roughly spherical clusters and is sensitive to scale — hence standardisation and block weighting. DBSCAN (density-based, finds outliers) was deliberately deferred because its eps/min_samples need careful tuning.

Streamlit and Plotly

What it is. Streamlit turns a Python script into an interactive web app: every widget interaction re-runs the script top to bottom; caching decorators avoid repeating expensive work. Plotly draws interactive (hover, zoom) charts in the browser.

How OntoRisk BI uses it. Single page: st.metric KPI cards, multiselect / date_input / radio / slider filters, st.dataframe with progress-bar columns for the rules, Plotly Express bar (OLAP) and scatter (clusters) charts, an expander with a SPARQL console.

Significance in the project. It delivers the decision-support layer with a few hundred lines of Python and no front-end code.

Good to know. Run with `streamlit run dashboard/app.py`; it serves on http://localhost:8501. Business logic sits in `logic.py` so it can be unit tested.

pytest, matplotlib, Git/GitHub

What it is. pytest is Python's de-facto test framework; matplotlib is the base plotting library; Git with GitHub hosts the code and supports branch-based collaboration.

How OntoRisk BI uses it. 17 tests cover the ontology, NER, longest-match tagging, the co-occurrence rule, RDF validity, severity bands, ETL quality checks, warehouse integrity, FP-Growth = Apriori, k-Means diagnostics, precision, dashboard KPIs and a full dashboard run. matplotlib renders the figures in outputs/figures. One Git branch per pipeline stage, merged into main after review (§4.8).

Significance in the project. Tests make each stage independently verifiable — the property the layered architecture was designed for.

Good to know. `python -m pytest` runs everything; the dashboard test uses Streamlit's official AppTest when Streamlit is installed.

6. Methodologies in more depth

6.1 The semantic-web stack

Layers, bottom-up: IRIs to name things → RDF to state facts as triples → RDFS/OWL to define the vocabulary (classes, properties, constraints) → SPARQL to query → reasoners to infer and check consistency. OntoRisk BI uses each layer minimally: IRIs under ontorisk.example.org, RDF triples from the co-occurrence rule, a six-class OWL ontology, SPARQL validation, and Protégé/HermiT for consistency checking.

6.2 ETL and data quality

Extract (read CSV) → Transform (clean, validate, enrich with NLP, compute measures, map to surrogate keys) → Load (dimensions before facts, then the bridge). Quality checks follow the report: empty/null text, duplicate ids and duplicate texts, unparseable dates, and post-load referential integrity (no fact row may point to a missing dimension row; every fact has triples; no null measures). Rejections are counted in `metrics.json`.

6.3 Rule-based relation extraction vs. learned extraction

Learned relation extraction (e.g. a fine-tuned transformer classifying entity pairs) needs annotated relation data. Co-occurrence within a bounded window is the established low-cost alternative: high recall, lower precision, fully transparent. Precision is then *measured*, not assumed — and that number conditions how much trust to put in everything downstream.

6.4 Association-rule mining, step by step

- Encode each event as a set of items; one-hot encode all baskets.
- FP-Growth: count item frequencies, drop infrequent items, sort items in each basket by frequency, insert baskets into a prefix tree (FP-tree); mine each item's conditional pattern base recursively to list all frequent itemsets.
- For each frequent itemset, split into antecedent ⇒ consequent and compute confidence and lift; keep rules above the thresholds; rank by lift.
- Interpretation: lift > 1 with good support and confidence indicates a recurring risk pattern worth monitoring; rules describe association, not causation.

6.5 Clustering, step by step

- Build the feature matrix; standardise each feature to mean 0, variance 1; weight one-hot and temporal blocks.
- For each k: run k-Means 10 times from different starts (k-means++), keep the lowest inertia; record inertia and silhouette.
- Pick k at the elbow; confirm with silhouette; refit; describe each cluster by size, average impact, dominant risk type, monsoon share and share of High-severity events; generate a readable label.
- Project to 2-D with PCA only for display — the clustering itself uses all features.

6.6 Evaluation metrics

- Precision = correct extracted events / all extracted events; Recall = true events found / all true events; F1 = harmonic mean.
- Association rules: support, confidence, lift thresholds (and leverage/conviction reported).
- Clustering: inertia (elbow) and silhouette coefficient, plus visual inspection of the scatter plot.

7. Results summary

Measure	Value
Articles (raw → clean)	243 → 240
Risk events extracted	387
By risk type	logistics 113, weather 86, policy 74, supplier 64, reputational 50
By sector	Supply Chain & Logistics 125, Manufacturing 118, Retail 67, Finance 63, General 14
RDF triples (total / generated)	6,068 / 5,605
SPARQL checks passed	6 / 6
Warehouse orphans / facts without triples	0 / 0
Frequent itemsets / rules	103 / 27
k-Means k / silhouette	5 / 0.239
Triple precision / recall / F1	72.4% / 92.6% / 0.812
NLP backend	spaCy 3.8.7 / en_core_web_sm 3.8.0
Warehouse engine	SQLite (fallback - DuckDB not installed)
Pipeline runtime	15.2 s

Note on the warehouse engine for these numbers. The run that produced this report was made on a machine where the DuckDB Python package could not be installed, so the pipeline used its built-in SQLite fallback with the *identical* schema and SQL. The same star schema, view and every query in `warehouse/olap_queries.sql` (ROLLUP, CUBE, the hybrid join) were separately loaded and run in the DuckDB 1.4.1 CLI without errors. After `pip install -r requirements.txt` on your laptop, `run_pipeline.py` uses DuckDB automatically and writes `warehouse/ontorisk.duckdb`; all counts and mining results are the same, because the warehouse engine does not affect them.

Key findings on this corpus. (1) Weather disruption is concentrated in the monsoon months and co-occurs with logistics disruption with lift above 2.5; (2) weather events are disproportionately High severity, especially for logistics and manufacturing firms; (3) policy events are typically Medium severity and come with supplier knock-on effects; (4) clustering separates high-impact monsoon weather events from low-impact off-season events. Because the corpus is synthetic these findings show the pipeline *can recover* planted patterns — on real news the same code reports whatever patterns exist.

8. How to run the project

8.1 One-time setup

```
# 1. unzip ontorisk-bi.zip and open a terminal in the ontorisk-bi folder
# 2. create and activate a virtual environment (Python 3.10 - 3.13)
python -m venv .venv
.venv\Scripts\activate            # Windows (PowerShell / cmd)
source .venv/bin/activate          # macOS / Linux

# 3. install the libraries and the spaCy English model
pip install -r requirements.txt
python -m spacy download en_core_web_sm
```

8.2 Build everything

```
python run_pipeline.py
# [hh:mm:ss] 1/8 corpus: 243 raw records (synthetic)
# [hh:mm:ss] 2/8 ontology: ... T-Box triples -> ontology/ontorisk.owl
# ...
# [hh:mm:ss] 8/8 evaluation: precision=... recall=...; figures -> outputs/figures
# Done. Launch the dashboard with:  streamlit run dashboard/app.py
```

8.3 Test, then open the dashboard

```
python -m pytest                    # all tests should pass
streamlit run dashboard/app.py      # browser opens at http://localhost:8501
```

8.4 Optional steps

- **Real news data**: `python run_pipeline.py --input path/to/news.csv` (no gold file → the manual review sheet is the precision check).
- **Bigger synthetic corpus**: `python run_pipeline.py --n 300`.
- **Protégé**: open `ontology/ontorisk.owl`.
- **DuckDB CLI**: `duckdb warehouse/ontorisk.duckdb < warehouse/olap_queries.sql`.
- **Change thresholds** (support, confidence, lift, k range, severity bands) in `src/ontorisk/config.py`; edit the vocabulary in `data/risk_vocabulary.json`.
- **Regenerate this PDF**: `python docs/build_explanation_pdf.py`.

8.5 Troubleshooting

Symptom	Fix
OSError: [E050] Can't find model 'en_core_web_sm'	python -m spacy download en_core_web_sm (inside the activated venv)
metrics.json says “rule-based fallback” or “SQLite”	spaCy / DuckDB not installed in the active environment — pip install -r requirements.txt, then re-run the pipeline
Dashboard: “Warehouse not found”	run python run_pipeline.py first
duckdb IOException: could not set lock	close other programs (DuckDB CLI, another Streamlit) using warehouse/ontorisk.duckdb, then re-run
'streamlit' is not recognized	activate the venv, or use python -m streamlit run dashboard/app.py
No rules shown in dashboard	selection has fewer than 25 events or thresholds too strict — widen filters or lower thresholds

9. Team allocation and timeline (Table 4.4, mapped to code)

Week	Member tasks	Code produced	Deliverable check
1	A — ontology in Protégé; B — dataset collection & cleaning; C — spaCy NER pipeline; D — DuckDB schema & ETL skeleton	ontology.py, corpus.py, etl.py, nlp.py, warehouse.py (DDL)	six-class ontology; corpus cleaned; NER on sample text; empty star schema
2	A + C — co-occurrence triples & ontology linking; B + D — ETL load & quality checks	extraction.py, semantic.py, warehouse.py (load, checks)	triples linked; SPARQL checks pass; warehouse populated and linked to triples
3	A + B — FP-Growth & k-Means; C + D — Streamlit dashboard; all — integration & testing	mining.py, evaluate.py, figures.py, dashboard/, tests/	rules & clusters evaluated; dashboard live; end-to-end tests pass; documentation & demo

Git workflow: one branch per stage (e.g. `feat/ontology`, `feat/ner`, `feat/warehouse`, `feat/mining`, `feat/dashboard`), merged to `main` by pull request after the tests pass.

10. Limitations and future scope

Limitations (from the report, confirmed by the build)

- Corpus of 150–300 articles is modest; the bundled corpus is synthetic, so the mined patterns demonstrate the method rather than real-world findings.
- Co-occurrence produces spurious triples (measured precision 72.4%).
- Compact ontology, two object properties, no causal taxonomy: the semantic layer records association, not causation.
- Entity and source are degenerate attributes — fine at this scale, not for hierarchical browsing.
- Batch processing of historical text, no live feeds; no supervised severity classification (no labels); no alerting layer.
- Impact score is a transparent heuristic, not a calibrated financial measure.

Future scope

- Pruned decision-tree classifier for severity once a labelled subset exists; DBSCAN with tuned parameters for outliers; rule-based alerting over mining outputs.
- Learned relation extraction (fine-tuned transformer) replacing the co-occurrence rule; then a causal relation taxonomy (causes, mitigates) in the ontology.
- Entity and source as full dimensions with SCD Type 2; real-time news / social-media ingestion; more sectors and geographies; SVM, ensembles, BIRCH, OPTICS on a larger labelled dataset.
- Entity linking to Wikidata / company registries; a triple store (Fuseki/GraphDB) with OWL reasoning.

11. Likely questions and model answers

What is the main contribution of OntoRisk BI?

Integration: a single open pipeline that connects text analytics, a semantic layer (ontology + RDF + SPARQL), a star-schema warehouse and data mining, ending in a dashboard. No single layer is novel; the end-to-end combination for SME risk intelligence is.

Why do you need both RDF and a star schema? Isn't that duplication?

They serve different questions. The RDF graph captures meaning and relationships and supports flexible, relationship-first SPARQL queries and validation against the ontology; the star schema gives fast, familiar OLAP aggregation for BI. The shared event_id and the bridge table keep the two consistent — this is the 'hybrid warehouse'.

How are relations extracted?

By a documented co-occurrence rule, not a trained model: an organisation, a location and a risk term in the same paragraph generate 'entity affects event' and 'event locatedIn location'. It is fast and auditable; its precision is measured.

What is the precision of your extraction and how did you measure it?

72.4% precision and 92.6% recall against ground-truth labels for every event in the corpus; a 50-triple random sample is also exported for manual review, as planned in the report.

Why spaCy and not a transformer like BERT?

Pre-trained spaCy pipelines give a good accuracy/cost balance, run on a laptop CPU and need no annotated data. A transformer would need fine-tuning data and GPU time we did not have in three weeks.

Why only six classes and two object properties?

A compact ontology is easier to populate consistently from automatic extraction. Defining properties such as 'causes' that the rule cannot populate faithfully would weaken the semantic layer.

Why is impact a data property and not a class?

Impact is a value describing one event (a score and a band), not a thing with its own identity and relationships, so modelling it as a data property is simpler and correct.

What does 'degenerate dimension' mean?

An attribute kept on the fact row without its own dimension table — here entity name and source id. They are never browsed hierarchically, so a dimension table would only add joins.

Why DuckDB instead of MySQL/PostgreSQL?

DuckDB is an embedded columnar OLAP engine: no server, one file, very fast aggregation, and it reads pandas DataFrames directly — ideal for a short laptop project.

Explain support, confidence and lift with one of your rules.

For "co_risk=logistics, sector=Supply Chain & Logistics, severity=High ⇒ risktype=weather": support 0.057 is the share of events containing both sides; confidence 0.88 is how often the consequent holds given the antecedent; lift 3.96 says that is 4.0× more than chance.

Why FP-Growth over Apriori?

FP-Growth avoids repeated candidate generation and database scans by mining a compressed FP-tree, so it scales better; on our data it returns exactly the same rules as Apriori, which we check.

How did you choose k for k-Means?

Elbow method on inertia for k = 2–10, detected automatically, validated with the silhouette score; chosen k = 5 with silhouette 0.239.

Why scale features and down-weight the one-hot block?

k-Means uses Euclidean distance, so unscaled features with large ranges dominate. Five one-hot columns for one attribute would also count five times; dividing the block by √5 gives it the weight of one feature.

Why not DBSCAN or decision trees?

DBSCAN needs careful eps/min_samples tuning on more data; decision trees need reliable severity labels that the corpus does not provide. Both are documented as future work.

Is the data real?

The bundled corpus is synthetic with fictional companies and real Indian places/seasonality, chosen so results are reproducible and precision can be measured exactly. The pipeline also runs unchanged on a real news CSV via --input.

How is the Risk Exposure Index computed?

Σ over events in view of impact score × severity weight (Low 1, Medium 2, High 3), shown with its share of total exposure.

What does the SPARQL validation check?

Six classes and two object properties exist; every event has a location and an affected entity; affects/locatedIn respect their domain and range; events per risk type are returned.

How does the dashboard stay responsive?

Data loading and rule mining are cached with st.cache_data, the RDF graph with st.cache_resource; rules are re-mined only when the filtered event set changes.

What happens with an article mentioning many companies?

The rule creates an event for each company–place–risk combination in the paragraph, which raises recall but lowers precision — the main limitation, quantified by the evaluation.

How would you scale this to thousands of articles a day?

Stream ingestion (e.g. RSS/APIs) into a queue, spaCy nlp.pipe batching, a triple store for the graph, incremental DuckDB loads or a cloud warehouse, scheduled re-mining, and alerting on new high-lift rules.

12. Glossary

Term	Meaning
Ontology	Formal definition of the concepts (classes) and relations (properties) of a domain.
Triple	Subject–predicate–object statement, the unit of RDF.
T-Box / A-Box	Terminology (the ontology schema) / Assertions (the instance facts).
IRI	Internationalised Resource Identifier — the global name of an RDF resource.
NER	Named Entity Recognition — finding names of organisations, places, dates in text.
Lemma	Dictionary form of a word (‘disrupted’ → ‘disrupt’).
Gazetteer	Lookup list of place names with their attributes.
Fact table / grain	Table of measurable events / what one row represents.
Dimension	Descriptive context for facts (time, location, risk type).
Surrogate key	System-generated integer key for a dimension row.
OLAP	Online Analytical Processing — roll-up, drill-down, slice, dice, pivot.
Itemset / transaction	Set of items / one basket (one event) in rule mining.
Support, confidence, lift	Frequency, conditional probability, and strength-versus-chance of a rule.
Inertia	Sum of squared distances of points to their cluster centroid.
Silhouette	Cluster quality from −1 (wrong) to 1 (well separated).

Term	Meaning
Precision / recall	Share of extracted items that are correct / share of true items extracted.

13. References

- R. Agrawal and R. Srikant, “Fast Algorithms for Mining Association Rules in Large Databases,” VLDB, 1994.
- J. Han, J. Pei and Y. Yin, “Mining Frequent Patterns without Candidate Generation,” ACM SIGMOD, 2000.
- J. MacQueen, “Some Methods for Classification and Analysis of Multivariate Observations,” 5th Berkeley Symposium, 1967.
- M. Ester, H.-P. Kriegel, J. Sander and X. Xu, “A Density-Based Algorithm for Discovering Clusters…,” KDD, 1996.
- J. R. Quinlan, C4.5: Programs for Machine Learning, Morgan Kaufmann, 1993.
- T. Berners-Lee, J. Hendler and O. Lassila, “The Semantic Web,” Scientific American, 2001.
- R. Kimball and M. Ross, The Data Warehouse Toolkit, 3rd ed., Wiley, 2013.
- W. H. Inmon, Building the Data Warehouse, 4th ed., Wiley, 2005.
- M. A. Musen, “The Protégé Project: A Look Back and a Look Forward,” AI Matters, 2015.
- P. J. Rousseeuw, “Silhouettes: a graphical aid to the interpretation and validation of cluster analysis,” J. Comput. Appl. Math., 1987.
- Documentation: spaCy (spacy.io), RDFLib (rdflib.readthedocs.io), W3C RDF/OWL/SPARQL 1.1, DuckDB (duckdb.org/docs), mlxtend (rasbt.github.io/mlxtend), scikit-learn (scikit-learn.org), Streamlit (docs.streamlit.io), Plotly (plotly.com/python).